

自然语言处理基础期中专题复习笔记

按考试范围重组的 FNLP 复习稿

自动整理于 2026-04-22

范围：分类、序列标注、N-gram、词表示、神经网络、CFG/PCFG、依存句法与典型分析算法

目录

1 使用说明与考试地图	7
1.1 考试范围的统一视角	7
1.2 先记输出对象，再记模型	7
1.3 考试答题总策略	8
2 专题一：基本分类问题的经典模型及评价方法	8
2.1 任务定义	8
2.2 分类问题的统一建模框架	9
2.3 朴素贝叶斯	9
2.3.1 基本思想	9
2.3.2 为什么这个推导考试常考	9
2.3.3 参数估计	10
2.3.4 优点与局限	10
2.4 对数线性模型 / 最大熵模型	11
2.4.1 从特征函数到概率	11
2.4.2 特征工程的意义	11
2.4.3 训练目标	11
2.5 生成式与判别式的统一比较	12

目录	2
2.6 分类问题的评价方法	12
2.6.1 Accuracy	12
2.6.2 Precision / Recall / F1	12
2.6.3 Macro / Micro	13
2.6.4 混淆矩阵	13
2.7 这一专题必须会的答题点	13
3 专题二：序列标注问题的经典模型、解码算法及评价方法	13
3.1 任务定义	13
3.2 为什么它不能简单看成逐词分类	14
3.3 常见标注方案：BIO / BIEO	14
3.4 HMM：经典生成式序列模型	14
3.4.1 基本假设	14
3.4.2 联合概率分解	14
3.4.3 训练	15
3.5 Viterbi 解码	15
3.5.1 为什么需要解码算法	15
3.5.2 状态定义	15
3.5.3 递推	15
3.5.4 复杂度	16
3.6 MEMM 与 label bias	16
3.7 线性链 CRF	16
3.7.1 打分函数	16
3.7.2 条件概率	17
3.7.3 为什么它优于逐位置分类	17
3.7.4 解码	17
3.8 序列标注的评价方法	17
3.8.1 Token-level accuracy	17
3.8.2 实体级 Precision / Recall / F1	17

目录	3
3.9 这一专题必须会的答题点	18
4 专题三：N-gram 语言模型的特点、训练与计算	18
4.1 语言模型的定义	18
4.2 N-gram 假设	18
4.3 N-gram 的训练	19
4.3.1 极大似然估计	19
4.3.2 句子概率计算	19
4.4 N-gram 的特点	19
4.4.1 优点	19
4.4.2 局限	20
4.5 平滑	20
4.5.1 加一平滑	20
4.6 交叉熵与困惑度	20
4.7 这一专题必须会的答题点	21
5 专题四：词汇表示的基本方法	21
5.1 为什么需要词表示	21
5.2 向量空间模型 (Vector Space Model)	21
5.2.1 基本思想	21
5.2.2 one-hot	22
5.3 TF-IDF	22
5.4 余弦相似度	22
5.5 共现矩阵与分布假说	22
5.6 PMI 与 PPMI	23
5.7 从稀疏分布式统计到稠密词向量	23
5.8 word2vec: CBOW 与 Skip-gram	23
5.8.1 CBOW	23
5.8.2 Skip-gram	24

5.9 这一专题必须会的答题点	24
6 专题五：典型神经网络模型及其在语言模型、分类问题上的应用	24
6.1 神经网络方法为什么会出现	24
6.2 神经网络分类器的最小结构	25
6.3 损失函数：交叉熵	25
6.4 前馈神经语言模型	25
6.5 RNN 语言模型	26
6.6 LSTM 的课程定位	26
6.7 神经网络在分类中的典型用法	26
6.7.1 Embedding + 平均池化 + MLP	26
6.7.2 CNN 文本分类	27
6.7.3 RNN / LSTM 文本分类	27
6.8 传统模型与神经模型比较	27
6.9 这一专题必须会的答题点	27
7 专题六：上下文无关文法、概率型上下文无关文法、依存句法分析	28
7.1 为什么需要句法结构	28
7.2 上下文无关文法 CFG	28
7.2.1 定义	28
7.2.2 为什么叫“上下文无关”	28
7.2.3 派生树与句法歧义	29
7.3 概率型上下文无关文法 PCFG	29
7.3.1 基本思想	29
7.3.2 树的概率	29
7.3.3 参数估计	29
7.3.4 PCFG 的局限	29
7.4 依存句法	30
7.4.1 基本单位	30

目录	5
7.4.2 几个核心概念	30
7.4.3 依存树的基本约束	30
7.4.4 投射性	30
7.5 成分句法与依存句法的比较	31
7.6 这一专题必须会的答题点	31
8 专题七：典型句法分析算法及评价方法	31
8.1 CFG/PCFG 的典型算法：CKY	31
8.1.1 适用条件	31
8.1.2 动态规划状态	31
8.1.3 初始化	32
8.1.4 递推	32
8.1.5 复杂度	32
8.2 依存句法的典型算法：Transition-based Parsing	32
8.2.1 状态表示	32
8.2.2 Arc-standard 动作	32
8.2.3 为什么需要 oracle	33
8.2.4 beam search	33
8.3 句法分析的评价方法	33
8.3.1 成分句法：Bracketing Precision / Recall / F1	33
8.3.2 依存句法：UAS / LAS	33
8.4 这一专题必须会的答题点	34
9 高频综合对比与易错点	34
9.1 高频综合对比	34
9.2 易错点清单	35
10 最后冲刺：公式速记与答题模板	35
10.1 最值得默写的公式	35
10.2 最稳妥的比较题模板	36

目 录	6
10.3 最稳妥的算法题模板	36
10.4 最稳妥的简答题模板	37
11 整理说明	37

1 使用说明与考试地图

这份笔记不是完整课程讲义的缩写版，而是按照期中考试范围重新组织的一份**题型复习稿**。它的目标有三个：

1. 帮你把题目一看到就能迅速判断“这属于哪一类模型、该写哪个公式、该用哪种评价指标”；
2. 把容易混的概念放在一起对照记忆，而不是按 lecture 顺序零散回看；
3. 把考试最常见的输出形式统一起来：**分类、序列、句子概率、树/图结构**。

1.1 考试范围的统一视角

用户给出的范围可以压缩成下面七个专题：

1. **基本分类问题**：输入一个对象，输出一个类别；
2. **序列标注问题**：输入一个词序列，输出等长标签序列；
3. **N-gram 语言模型**：给整句赋概率；
4. **词汇表示**：把离散词变成向量，并定义相似度；
5. **典型神经网络模型**：把表示学习和预测统一在一个可训练框架中；
6. **CFG / PCFG / 依存句法**：输出句法结构；
7. **句法分析算法**：给定文法或动作系统，真正算出最优结构。

1.2 先记输出对象，再记模型

一门课里模型很多，但其实考试时最快的判断方法不是背模型名字，而是先问自己：

- 输出是**单个标签**吗？那大概率是分类；
- 输出是**每个位置一个标签**吗？那是序列标注；
- 输出是**整句概率**吗？那是语言模型；
- 输出是**树结构或依存弧**吗？那是句法分析；
- 输出是**向量**吗？那是表示学习问题。

一旦输出对象明确，后面的很多问题都会自动跟出来：

- 建模目标写成 $p(y | x)$ 、 $p(x, y)$ 、 $p(w_{1:T})$ 还是 $p(T, x)$ ；
- 解码时求单点 $\arg \max$ 还是整序列/整棵树的 $\arg \max$ ；
- 评价时看 accuracy、F1、perplexity，还是 UAS/LAS。

1.3 考试答题总策略

如果题目是“定义/比较题” 优先按“建模对象 → 核心假设 → 训练方式 → 解码方式 → 评价方式 → 优缺点”的顺序回答。

如果题目是“公式推导题” 优先按“原始目标 → 公式展开 → 近似/独立性假设 → 忽略常数项 → 最终可计算形式”的顺序写。

如果题目是“算法题” 优先写三件事：

1. 状态或数据结构是什么；
2. 递推/动作规则是什么；
3. 最终答案如何恢复，复杂度大概从哪里来。

2 专题一：基本分类问题的经典模型及评价方法

2.1 任务定义

分类问题的标准形式是：给定输入 x ，从标签集合 $\mathcal{Y}$ 中预测一个标签 y 。其最基本的决策形式是

$$y^* = \arg \max_{y \in \mathcal{Y}} \text{Score}(x, y).$$

如果分数来自条件概率，那么就写成

$$y^* = \arg \max_{y \in \mathcal{Y}} p(y | x).$$

课程中的典型例子

- 词义消歧：给定上下文，判断目标词是哪一个义项；

- 文本分类：给定句子或文档，判断主题、情感或类别；
- 二分类与多分类：输出可以是两个类别，也可以是多个类别。

2.2 分类问题的统一建模框架

从课程角度看，分类模型其实都在做同一件事：给每个候选标签打分。它们的差异主要来自两个维度：

1. **打分依据**：是建模联合概率、条件概率，还是直接学一个线性/非线性评分函数；
2. **输入表示**：是人工特征、离散词袋，还是连续向量。

因此，我们常把经典模型分成两大类：

- **生成式模型**：先建模“这个标签会怎样生成当前输入”，再用贝叶斯公式分类；
- **判别式模型**：直接建模“在给定输入时哪个标签最可能”。

2.3 朴素贝叶斯

2.3.1 基本思想

朴素贝叶斯 (Naive Bayes) 是假设在给定类别 y 的条件下，各个输入特征彼此独立。于是

$$p(x | y) = \prod_{i=1}^m p(x_i | y).$$

利用贝叶斯公式，

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)}.$$

由于对固定输入 x 而言， $p(x)$ 对所有类别相同，所以分类决策化为

$$y^* = \arg \max_y p(y) \prod_i p(x_i | y).$$

2.3.2 为什么这个推导考试常考

因为它几乎完整展示了分类推导题的标准套路：

1. 原始目标是 $\arg \max_y p(y | x)$ ；
2. 用贝叶斯公式把它转成 $\arg \max_y p(x | y)p(y)$ ；

3. 再用条件独立假设把 $p(x | y)$ 分解成局部项的乘积；
4. 最终得到可以通过计数训练的形式。

2.3.3 参数估计

若用离散特征统计极大似然参数，则

$$p(y) = \frac{\text{Count}(y)}{N}, \quad p(x_i = v | y) = \frac{\text{Count}(x_i = v, y)}{\sum_{v'} \text{Count}(x_i = v', y)}.$$

平滑的必要性 若某个特征值在某个类里没出现过，那么极大似然会得到概率 0，整条乘积就会被“乘没”。所以实际使用时经常要加平滑，例如加一平滑：

$$p(x_i = v | y) = \frac{\text{Count}(x_i = v, y) + 1}{\sum_{v'} \text{Count}(x_i = v', y) + |V_i|}.$$

2.3.4 优点与局限

优点：

- 训练简单，计数即可；
- 在小数据和高维稀疏特征场景下常常 surprisingly effective；
- 推理速度快。

局限：

- 条件独立假设在语言数据中往往不成立；
- 特征交互关系无法直接表达；
- 得到的概率常常校准较差。

为什么“明知假设不对”还能用 这是很典型的简答题。可以从三点回答：

1. 分类只需要排序足够好，不要求概率完美真实；
2. 即使特征相关，它们对类别的偏向方向仍然可能一致；
3. 强假设降低了参数复杂度，在小数据下反而更稳。

2.4 对数线性模型 / 最大熵模型

2.4.1 从特征函数到概率

对数线性模型不再先建模 $p(x | y)$ ，而是直接给每个 (x, y) 定义特征函数 $\mathbf{f}(x, y)$ 和权重向量 $\mathbf{w}$ ，其条件概率写为

$$p(y | x) = \frac{\exp(\mathbf{w} \cdot \mathbf{f}(x, y))}{\sum_{y'} \exp(\mathbf{w} \cdot \mathbf{f}(x, y'))}.$$

怎么理解这条式子 可以把它分成两部分：

- 分子 $\exp(\mathbf{w} \cdot \mathbf{f}(x, y))$ ：给标签 y 的未归一化分数；
- 分母：对所有候选标签做归一化，让结果成为合法概率分布。

2.4.2 特征工程的意义

对数线性模型的威力来自“特征很自由”。我们可以把很多语言现象写成指示函数，比如：

$$f_k(x, y) = \begin{cases} 1, & \text{若输入中出现某词且标签为 } y, \\ 0, & \text{否则。} \end{cases}$$

因此它比朴素贝叶斯更灵活，因为它不要求特征彼此条件独立，只要你能设计出有用特征，就能让模型直接学权重。

2.4.3 训练目标

训练时通常最大化条件对数似然：

$$\mathcal{L}(\mathbf{w}) = \sum_{n=1}^N \log p(y^{(n)} | x^{(n)}).$$

为了防止过拟合，常加入 L_2 正则：

$$\mathcal{J}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) - \lambda \|\mathbf{w}\|_2^2.$$

与朴素贝叶斯的核心差异

- 朴素贝叶斯是**生成式**，最大熵是**判别式**；
- 朴素贝叶斯依赖局部条件独立，最大熵不需要；

- 朴素贝叶斯参数能直接计数，最大熵通常要做数值优化；
- 最大熵更适合融合重叠、相关、复杂的特征。

2.5 生成式与判别式的统一比较

比较维度	生成式模型	判别式模型
直接建模对象	$p(x, y)$ 或 $p(x y)p(y)$	$p(y x)$ 或直接打分函数
典型模型	朴素贝叶斯、HMM、PCFG	最大熵、CRF、神经分类器
优点	结构清晰，能解释数据生成过程，常适合小数据	特征自由，分类边界更直接，预测效果通常更强
局限	假设较强，难表达复杂依赖	常需要优化算法，训练更重
常见题型	公式推导、条件独立假设	softmax、特征权重、条件似然

2.6 分类问题的评价方法

2.6.1 Accuracy

最基础的分类评价是准确率：

$$\text{Accuracy} = \frac{\text{预测正确的样本数}}{\text{总样本数}}.$$

它适合类别分布较均衡、并且所有错误代价差不多的场景。

2.6.2 Precision / Recall / F1

当正负样本不平衡时，单看 accuracy 容易误导。于是要看：

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = \frac{2PR}{P + R}.$$

直觉解释

- Precision 高：你报出来的“正例”大多真的对；
- Recall 高：真正的正例里，你找回来了大部分；
- F1：把 precision 和 recall 做调和平均，防止其中一项特别差时被另一项掩盖。

2.6.3 Macro / Micro

多分类里还经常问 macro 与 micro:

- **Macro-F1**: 先对每个类分别算 F1, 再平均, 对小类更敏感;
- **Micro-F1**: 把所有类的 TP/FP/FN 合在一起再算, 更接近总体样本层面表现。

2.6.4 混淆矩阵

混淆矩阵的重要作用是看模型**错在哪儿**。对考试而言, 只要记住: 它不是额外的指标, 而是把不同类别之间的误分类模式显式展示出来。

2.7 这一专题必须会的答题点

1. 能从 $\arg \max p(y | x)$ 推导到朴素贝叶斯决策式;
2. 能解释为什么需要平滑;
3. 能写出最大熵/对数线性模型的 softmax 形式;
4. 能比较生成式与判别式;
5. 能区分 accuracy、precision、recall、F1 在什么情况下更有意义。

3 专题二：序列标注问题的经典模型、解码算法及评价方法

3.1 任务定义

序列标注 (sequence labeling) 的输入是序列 $x_{1:T}$, 输出是同长度标签序列 $y_{1:T}$ 。典型任务包括:

- 词性标注;
- 命名实体识别;
- 分词或 chunking。

形式上, 目标是

$$y_{1:T}^* = \arg \max_{y_{1:T}} p(y_{1:T} | x_{1:T})$$

或者在生成式模型里写成

$$y_{1:T}^* = \arg \max_{y_{1:T}} p(x_{1:T}, y_{1:T}).$$

3.2 为什么它不能简单看成逐词分类

如果把每个词独立分类，就会丢掉标签之间的相邻约束。例如实体识别中，I-PER 通常不能无缘无故出现在句首；词性标注中，某些标签转移明显更常见。因此序列标注问题的核心难点不在“每个位置预测什么”，而在“**整条标签序列要不要一致**”。

3.3 常见标注方案：BIO / BIEO

- B: 实体开始；
- I: 实体内部；
- O: 不属于任何目标实体；
- E: 实体结束（若采用 BIEO）；
- S: 单字/单词实体（若采用 BIOES）。

为什么标签方案会影响模型 因为标签方案实际上把结构信息编码进了局部状态集合里。标签越细，局部约束表达越充分，但状态空间也会变大。

3.4 HMM：经典生成式序列模型

3.4.1 基本假设

隐马尔可夫模型（Hidden Markov Model, HMM）有两个关键假设：

1. **一阶马尔可夫假设**：当前标签只依赖前一个标签；
2. **输出独立假设**：当前位置的观测只依赖当前标签。

3.4.2 联合概率分解

若标签序列为 $y_{1:T}$ ，观测序列为 $x_{1:T}$ ，则

$$p(x_{1:T}, y_{1:T}) = p(y_1)p(x_1 | y_1) \prod_{t=2}^T p(y_t | y_{t-1})p(x_t | y_t).$$

也常写成

$$p(x_{1:T}, y_{1:T}) = \pi_{y_1} b_{y_1}(x_1) \prod_{t=2}^T a_{y_{t-1}, y_t} b_{y_t}(x_t),$$

其中

- π_i 是初始状态概率；
- a_{ij} 是状态转移概率；
- $b_j(x_t)$ 是发射概率。

3.4.3 训练

若训练语料中标签已知，那么参数可以直接计数估计：

$$a_{ij} = \frac{\text{Count}(y_{t-1} = i, y_t = j)}{\text{Count}(y_{t-1} = i)}, \quad b_j(w) = \frac{\text{Count}(y_t = j, x_t = w)}{\text{Count}(y_t = j)}.$$

HMM 在课上的意义 它是第一个真正把“单点分类”提升到“全序列建模”的模型。它告诉我们：只要标签之间有依赖，我们就不能再只对每个位置各自做 softmax，而要对整条序列求最优。

3.5 Viterbi 解码

3.5.1 为什么需要解码算法

对长度为 T 、标签集大小为 K 的序列，若暴力枚举所有标签序列，复杂度约为 K^T ，显然不可行。Viterbi 的作用就是用动态规划把“指数级枚举”变成“多项式时间递推”。

3.5.2 状态定义

令

$$\delta_t(j) = \max_{y_{1:t-1}} p(y_{1:t-1}, y_t = j, x_{1:t}).$$

它表示：在第 t 个位置以标签 j 结尾时，前缀序列能达到的最大联合概率。

3.5.3 递推

初始化：

$$\delta_1(j) = \pi_j b_j(x_1).$$

递推：

$$\delta_t(j) = \max_i \delta_{t-1}(i) a_{ij} b_j(x_t).$$

终止：

$$\max_j \delta_T(j).$$

同时保存回溯指针

$$\psi_t(j) = \arg \max_i \delta_{t-1}(i) a_{ij},$$

最后沿着指针逆推恢复最优路径。

3.5.4 复杂度

一阶 HMM 的 Viterbi 时间复杂度通常是 $O(TK^2)$ ，空间复杂度若保留整张表是 $O(TK)$ 。

考试写法提醒 算法题一定别只写“用 Viterbi”。要把状态定义、初始化、递推、回溯四部分写全。

3.6 MEMM 与 label bias

最大熵马尔可夫模型 (MEMM) 把局部分类器和状态转移结合起来，形式上更灵活，但会有著名的 **label bias problem**：

- 每个前一状态的出边概率单独归一化；
- 出边少的状态可能因为归一化方式而被系统性偏好；
- 结果是模型容易过分依赖状态转移结构，而忽略全局观测证据。

为什么这会引出 CRF 因为我们想保留“条件建模、特征自由”的优势，但又不想像 MEMM 那样做局部归一化，于是自然导向全局归一化的 CRF。

3.7 线性链 CRF

3.7.1 打分函数

对输入 x 与标签序列 y ，线性链 CRF 定义全局分数

$$\text{Score}(x, y) = \sum_{t=1}^T \left(A_{y_{t-1}, y_t} + P_t(y_t) \right),$$

其中

- $A_{i,j}$ 是标签转移分数；
- $P_t(y_t)$ 是位置 t 上标签 y_t 的发射或局部打分。

3.7.2 条件概率

$$p(y \mid x) = \frac{\exp(\text{Score}(x, y))}{\sum_{y'} \exp(\text{Score}(x, y'))}.$$

3.7.3 为什么它优于逐位置分类

因为它不是对每个位置单独做 softmax，而是对**整条标签序列**做全局归一化，从而能表达标签转移约束并缓解 label bias。

3.7.4 解码

CRF 解码仍然要求

$$y^* = \arg \max_y \text{Score}(x, y),$$

因此仍可用与 Viterbi 类似的动态规划思想完成。

3.8 序列标注的评价方法

3.8.1 Token-level accuracy

若任务更接近词性标注，常用 token accuracy：

$$\text{Acc} = \frac{\text{标签预测正确的位置数}}{\text{总位置数}}.$$

3.8.2 实体级 Precision / Recall / F1

若任务是命名实体识别，更常用**实体级** P/R/F1，而不是逐 token accuracy。原因是：

- 实体边界必须整体正确；
- 只对了一半边界，通常不能算对；
- 因此以完整 span 为单位评价更合理。

一个常见坑 如果模型把 New York University 预测成 New York，token accuracy 可能不低，但实体级 F1 会真实反映这种边界错误。

3.9 这一专题必须会的答题点

1. 能说清序列标注为何不同于独立分类；
2. 能写出 HMM 的联合概率分解；
3. 能完整写出 Viterbi 的状态和递推；
4. 能解释 MEMM 的 label bias；
5. 能写出 CRF 的全局归一化形式；
6. 能区分 token accuracy 与实体级 F1。

4 专题三：N-gram 语言模型的特点、训练与计算

4.1 语言模型的定义

语言模型 (Language Model, LM) 的目标是给一个词序列赋概率：

$$p(w_{1:T}).$$

利用链式法则，

$$p(w_{1:T}) = \prod_{t=1}^T p(w_t \mid w_{1:t-1}).$$

这条公式为什么是全章起点 因为后面所有语言模型，不论是 N-gram、神经 LM 还是 RNN/Transformer，本质都在回答同一个问题：

如何近似 $p(w_t \mid w_{1:t-1})$?

4.2 N-gram 假设

N-gram 模型用一个强近似把长历史截断：

$$p(w_t \mid w_{1:t-1}) \approx p(w_t \mid w_{t-n+1:t-1}).$$

常见特例

- unigram: $p(w_t)$;
- bigram: $p(w_t \mid w_{t-1})$;
- trigram: $p(w_t \mid w_{t-2}, w_{t-1})$ 。

4.3 N-gram 的训练

4.3.1 极大似然估计

对历史 h 和下一个词 w , N-gram 的极大似然估计为

$$p(w | h) = \frac{\text{Count}(h, w)}{\text{Count}(h)}.$$

bigram 的写法

$$p(w_t | w_{t-1}) = \frac{\text{Count}(w_{t-1}, w_t)}{\text{Count}(w_{t-1})}.$$

trigram 的写法

$$p(w_t | w_{t-2}, w_{t-1}) = \frac{\text{Count}(w_{t-2}, w_{t-1}, w_t)}{\text{Count}(w_{t-2}, w_{t-1})}.$$

4.3.2 句子概率计算

若是 bigram, 并在句子两端加入开始/结束符, 则句子

$$\langle s \rangle w_1 w_2 \cdots w_T \langle /s \rangle$$

的概率为

$$p(w_{1:T}, \langle /s \rangle | \langle s \rangle) = \prod_{t=1}^{T+1} p(w_t | w_{t-1}),$$

其中把 w_{T+1} 视为结束符。

考试中的“计算题”通常怎么出 一般会直接给你若干计数, 让你:

1. 算某个 N-gram 概率;
2. 算一句话的整体概率;
3. 有时再顺手算 log 概率或 perplexity。

4.4 N-gram 的特点

4.4.1 优点

- 简单直观, 可直接计数;
- 训练和解释都非常清楚;
- 是理解语言模型思想的最好起点。

4.4.2 局限

- 只能看到固定长度上下文；
- 数据稀疏问题严重；
- 无法自然泛化到未见组合；
- 参数量随 n 和词表扩大迅速膨胀。

一个非常高频的直觉题 “为什么 N-gram 会遇到数据稀疏？”因为语言组合空间增长极快。即使单词都见过，多个词拼起来的长片段也可能从未在训练集共同出现。

4.5 平滑

若某个 N-gram 从未出现，MLE 给出的概率就是 0，这会导致整句概率也变成 0。平滑的核心目标就是：

- 给未见事件留出一些概率质量；
- 让模型面对新句子时不至于完全崩掉。

4.5.1 加一平滑

最简单的形式是

$$p_{\text{add-1}}(w | h) = \frac{\text{Count}(h, w) + 1}{\text{Count}(h) + |V|}.$$

为什么它常被讲，但实际不一定最好 因为它思路简单，最适合教学；但真实场景里它往往会对高频事件惩罚过重、对低频事件补得过多。所以课程上更重要的是理解“平滑为什么存在”，而不是死背某一个最优公式。

4.6 交叉熵与困惑度

测试集平均负对数似然可写作

$$H = -\frac{1}{T} \sum_{t=1}^T \log p(w_t | h_t).$$

若用自然对数，则困惑度为

$$\text{PPL} = \exp(H) = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p(w_t | h_t)\right).$$

怎么理解 perplexity 它可以粗略理解为：模型在每一步平均还剩多少“等可能候选”的不确定性。PPL 越低，说明模型给真实词分配的概率越高。

为什么 PPL 适合语言模型 因为语言模型关心的是**概率分布质量**，而不是单次分类对错。语言模型哪怕给正确词最高分，也还要看它到底给了多高的概率。

4.7 这一专题必须会的答题点

1. 能从链式法则写出整句概率分解；
2. 能写出 bigram / trigram 的 MLE 公式；
3. 能根据计数算条件概率与句子概率；
4. 能解释数据稀疏与平滑；
5. 能写出困惑度公式并解释其含义。

5 专题四：词汇表示的基本方法

5.1 为什么需要词表示

机器无法直接理解“词”的语义，只能接收数值。词表示的目标就是把离散词变成计算对象，并让“相似词”在表示空间中体现出某种接近关系。

两条主线

- **稀疏表示**：高维、可解释，但泛化弱；
- **稠密表示**：低维、可学习、泛化强，但解释性较弱。

5.2 向量空间模型 (Vector Space Model)

5.2.1 基本思想

把词、句子或文档表示为一个向量，然后在向量空间里比较它们的相似度。最原始的做法往往是 bag-of-words：

$$\mathbf{x} = (x_1, x_2, \dots, x_{|V|}),$$

其中每一维对应词表中的一个词。

5.2.2 one-hot

词的最基础离散表示是 one-hot:

$$\mathbf{e}_i = (0, \dots, 0, 1, 0, \dots, 0).$$

它的优点是唯一、清晰，但问题也很明显：

- 任意两个不同词的距离都差不多；
- 不能表达语义相似性；
- 维度等于词表大小，高而稀疏。

5.3 TF-IDF

对文档表示而言，最常见的经典加权方案是 TF-IDF。若词 t 在文档 d 中的词频为 $\text{tf}(t, d)$ ，文档频率为 $\text{df}(t)$ ，总文档数为 N ，则常见定义为

$$\text{idf}(t) = \log \frac{N}{\text{df}(t)}, \quad \text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t).$$

直觉

- 在某一篇文章里出现得多：说明对这篇文档可能重要；
- 在所有文档里到处都出现：说明区分性弱，权重应降低。

5.4 余弦相似度

向量空间模型里最常见的相似度计算是余弦相似度：

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}.$$

为什么经常用 cosine 而不是直接点积 因为余弦相似度消除了向量长度影响，更强调方向是否相近。这在文本长度差异明显时尤其有用。

5.5 共现矩阵与分布假说

分布假说的核心是：一个词的意义由它所处的上下文决定。于是我们可以统计词与上下文共现，构造共现矩阵 X ，其中

$$X_{ij} = \text{Count}(\text{词}i \text{ 与上下文}j \text{ 共现}).$$

上下文如何定义 常见方式有：

- 固定窗口内相邻词；
- 同一文档；
- 某种句法邻接关系。

5.6 PMI 与 PPMI

为了衡量词与上下文的关联强度，常用点互信息：

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)}.$$

若为负值则截断为 0，就得到 PPMI：

$$\text{PPMI}(w, c) = \max(\text{PMI}(w, c), 0).$$

怎么理解 PMI 若 w 与 c 独立，则 $P(w, c) = P(w)P(c)$ ，PMI 为 0；若它们比独立时更常一起出现，PMI 为正；越大说明关联越强。

一个常考直觉题：为什么 PMI 偏爱低频词 因为当某词本身很少出现时，只要它和某上下文偶然共现几次，分母 $P(w)P(c)$ 可能非常小，从而把 PMI 抬得很高。所以实际使用时常配合频次截断或 PPMI。

5.7 从稀疏分布式统计到稠密词向量

共现矩阵、TF-IDF、PPMI 等都还是**显式统计表示**。它们维度高、通常稀疏。后来的表示学习进一步追求：

- 维度更低；
- 能自动学习；
- 能把语义相似性编码进连续空间。

5.8 word2vec: CBOW 与 Skip-gram

5.8.1 CBOW

CBOW 用上下文预测中心词：

$$p(w_t \mid \text{context}(w_t)).$$

它更像一个局部分类问题：把周围词的表示汇总起来，预测当前目标词。

5.8.2 Skip-gram

Skip-gram 反过来，用中心词预测上下文词：

$$\prod_{c \in \text{context}(w_t)} p(c | w_t).$$

它们和传统向量空间模型的关系 可以把它们看作一种“通过预测任务来学习词向量”的方式。与手工统计共现相比，它们让词表示成为一个可训练参数，而非静态统计量。

5.9 这一专题必须会的答题点

1. 能说明 one-hot 的局限；
2. 能写出 TF-IDF 与 cosine 相似度；
3. 能解释分布假说；
4. 能写出 PMI/PPMI 并解释其含义；
5. 能说清显式共现表示与稠密词向量的关系；
6. 能区分 CBOW 和 Skip-gram 的预测方向。

6 专题五：典型神经网络模型及其在语言模型、分类问题上的应用

6.1 神经网络方法为什么会出现

传统模型要么依赖强假设，要么高度依赖人工特征工程。神经网络的核心优势在于：

- 能把表示学习和任务预测放在一个统一框架中端到端训练；
- 能通过非线性变换表达更复杂的模式；
- 能在参数共享下实现泛化。

6.2 神经网络分类器的最小结构

最基本的神经分类器通常包含三层思想：

1. **Embedding 层**：把离散输入映射成稠密向量；
2. **编码层**：通过线性层、卷积、循环或注意力整合信息；
3. **输出层**：通过 softmax 预测类别。

若句子表示为向量 $\mathbf{h}$ ，则分类分布常写为

$$p(y | x) = \text{softmax}(W\mathbf{h} + \mathbf{b}).$$

6.3 损失函数：交叉熵

神经分类器最常见的训练目标是交叉熵损失。对单个样本，

$$\mathcal{L} = -\log p(y^{\text{gold}} | x).$$

对整个训练集则求和或求平均。

这和最大熵模型的关系 本质上非常近。只不过最大熵模型通常把 $\mathbf{h}$ 看成手工特征线性组合，而神经网络把 $\mathbf{h}$ 变成可学习的非线性表示。

6.4 前馈神经语言模型

前馈神经语言模型（NNLM）的基本思路是：

1. 取固定长度历史窗口；
2. 查询窗口中各词的 embedding；
3. 拼接或聚合后送入前馈网络；
4. 预测下一个词的概率分布。

形式上可写成

$$\mathbf{h} = g([\mathbf{e}(w_{t-n+1}); \cdots ; \mathbf{e}(w_{t-1})]), \quad p(w_t | h_t) = \text{softmax}(W\mathbf{h} + \mathbf{b}).$$

它相对 N-gram 的改进

- 仍看固定窗口，但不再完全依赖离散计数；

- 相似词可以共享向量空间，从而缓解数据稀疏；
- 未见过的组合也可能因为词向量接近而得到合理概率。

6.5 RNN 语言模型

RNN 的关键改进是不再把历史限制为固定窗口，而是递归维护一个隐藏状态：

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{e}(w_t)).$$

预测下一个词时用

$$p(w_{t+1} \mid w_{1:t}) = \text{softmax}(W\mathbf{h}_t + \mathbf{b}).$$

它解决了什么问题 理论上，RNN 可以把前面所有历史压缩进隐藏状态里，比 N-gram 与前馈 NNLM 更适合建模长程依赖。

它又遇到了什么问题 基础 RNN 在长序列上会遇到梯度消失/爆炸，因此后续发展出 LSTM/GRU，通过门控机制改善长期记忆。

6.6 LSTM 的课程定位

考试通常不要求你详细推门控公式，但应知道：

- LSTM 通过输入门、遗忘门、输出门控制信息流；
- 它比普通 RNN 更适合长距离依赖；
- 在语言模型、序列标注、文本分类中都非常常见。

6.7 神经网络在分类中的典型用法

6.7.1 Embedding + 平均池化 + MLP

这是一种最基础但很实用的文本分类方法：

1. 先把句子中每个词嵌入成向量；
2. 做平均或求和得到句子表示；
3. 送入 MLP + softmax 输出类别。

优点 简单、快、稳定，是“神经版词袋模型”。

6.7.2 CNN 文本分类

CNN 通过卷积核提取局部 n-gram 模式，然后通过池化得到句子表示，适合捕捉局部关键词或短语模式。

6.7.3 RNN / LSTM 文本分类

RNN/LSTM 更适合编码顺序信息。常见做法是：

- 取最后一个隐藏状态做分类；
- 或对所有时刻隐藏状态做池化/注意力聚合。

6.8 传统模型与神经模型的比较

比较维度	传统统计模型	神经网络模型
表示方式	人工特征、离散统计量	可学习稠密向量
依赖建模	常靠显式假设或手工设计	靠参数共享和非线性自动学习
优点	可解释、训练常更直接	泛化强、性能高、端到端
局限	特征工程重，稀疏严重	数据和算力需求更高，可解释性较弱

6.9 这一专题必须会的答题点

1. 能说出 embedding、编码层、softmax 的基本作用；
2. 能写出交叉熵损失；
3. 能解释前馈 NNLM 与 N-gram 的联系和区别；
4. 能说明 RNN/LSTM 为什么适合语言模型；
5. 能列举神经网络在分类中的几种基本结构。

7 专题六：上下文无关文法、概率型上下文无关文法、依存句法分析

7.1 为什么需要句法结构

到分类、序列标注这一步，我们输出的仍然只是标签或标签序列。但句子真正的组合关系往往是层次化的。例如：

old men and women

到底是“老年男性和女性”，还是“老年男性和老年女性”？这类歧义需要结构来表达。

7.2 上下文无关文法 CFG

7.2.1 定义

上下文无关文法（Context-Free Grammar, CFG）通常写成四元组

$$G = (N, \Sigma, R, S),$$

其中

- N ：非终结符集合；
- Σ ：终结符集合；
- R ：产生式规则集合；
- S ：开始符号。

规则一般形如

$$A \rightarrow \alpha,$$

其中 $A \in N$ ， α 是终结符与非终结符串。

7.2.2 为什么叫“上下文无关”

因为能否用规则 $A \rightarrow \alpha$ 替换，只看当前符号是不是 A ，而不看它左右两边处于什么上下文。

7.2.3 派生树与句法歧义

同一个句子可能对应多棵不同的分析树，这就叫句法歧义。CFG 能表达“哪些结构可能”，但不能自然表达“哪种结构更可能”。

7.3 概率型上下文无关文法 PCFG

7.3.1 基本思想

PCFG 在每条 CFG 规则上附加一个概率。例如

$$A \rightarrow \beta \quad [p].$$

且对同一左部 A 的所有规则，其概率和为 1：

$$\sum_{\beta} p(A \rightarrow \beta) = 1.$$

7.3.2 树的概率

若一棵分析树 T 使用了一系列规则，则在 PCFG 中，树概率通常写为这些规则概率的乘积：

$$p(T) = \prod_{r \in T} p(r).$$

为什么这很重要 它让“句法分析”从“找任意一棵合法树”变成“找概率最大的合法树”。

7.3.3 参数估计

若训练数据是已标注的树库，则 PCFG 的极大似然估计很直接：

$$p(A \rightarrow \beta) = \frac{\text{Count}(A \rightarrow \beta)}{\sum_{\beta'} \text{Count}(A \rightarrow \beta')}.$$

7.3.4 PCFG 的局限

虽然 PCFG 比 CFG 多了概率信息，但它仍带有较强独立性假设：一个非终结符如何展开，只看它本身，不看更大上下文。因此它对真实语言中的长距离依赖、词汇化现象表达有限。

7.4 依存句法

7.4.1 基本单位

依存句法不再强调短语成分，而强调词与词之间的支配关系。每条弧通常写成

$$\text{head} \rightarrow \text{dependent}.$$

7.4.2 几个核心概念

- **head**: 中心词;
- **dependent**: 依附于 head 的词;
- **root**: 全句根节点;
- **label**: 依存关系类型，如主谓、动宾、定中等。

7.4.3 依存树的基本约束

典型依存树要求:

1. 除 root 外每个词有且仅有一个 head;
2. 整体连通;
3. 无环。

7.4.4 投射性

若依存弧画在句子上方且没有交叉，则称为投射。很多经典转移系统先假设投射性，因为这样算法更简单。

7.5 成分句法与依存句法的比较

比较维度	成分句法	依存句法
结构单位	短语/非终结符节点	词与词之间的依存弧
输出形式	树形分层结构	以词为节点的有向树
优势	层次结构清晰，适合短语分析	更贴近谓词-论元关系，跨语言常更实用
典型模型	CFG、PCFG	transition-based、graph-based 依存解析
评价指标	Bracketing P/R/F1	UAS/LAS

7.6 这一专题必须会的答题点

- 1. 能定义 CFG 四元组；
- 2. 能解释 CFG 与 PCFG 的区别；
- 3. 能写出 PCFG 的树概率与 MLE；
- 4. 能说明依存句法中 head、dependent、root 的含义；
- 5. 能比较成分句法与依存句法的结构差异。

8 专题七：典型句法分析算法及评价方法

8.1 CFG/PCFG 的典型算法：CKY

8.1.1 适用条件

CKY（也常写作 CYK/CKY）通常要求文法先转成 Chomsky Normal Form，即规则大致形如

$$A \rightarrow BC \quad \text{或} \quad A \rightarrow w.$$

8.1.2 动态规划状态

令

$$\text{Chart}[i, j, A]$$

表示：非终结符 A 能否生成区间 $[i, j]$ 的子串，或者在概率版本中表示“ A 生成该区间的最大概率”。

8.1.3 初始化

对每个单词位置 i ，若有语法规则 $A \rightarrow w_i$ ，则填表

$$\text{Chart}[i, i, A] = p(A \rightarrow w_i)$$

或在布尔识别版本里记为 true。

8.1.4 递推

对任意区间 $[i, j]$ ，枚举切分点 k 与规则 $A \rightarrow BC$ ：

$$\text{Chart}[i, j, A] = \max_{k, A \rightarrow BC} p(A \rightarrow BC) \cdot \text{Chart}[i, k, B] \cdot \text{Chart}[k + 1, j, C].$$

怎么理解 CKY 它不是在“猜整棵树”，而是在“先算短区间，再拼长区间”。这就是标准区间动态规划思想。

8.1.5 复杂度

若句长为 n ，非终结符数量约为 $|N|$ ，则 CKY 的经典复杂度通常写成 $O(n^3|N|^3)$ 或不同实现下写成近似立方级别。考试里知道它是**立方级区间 DP** 就够用了。

8.2 依存句法的典型算法：Transition-based Parsing

8.2.1 状态表示

经典 transition-based parser 的状态通常由三部分组成：

- **Stack**：已经部分处理的词；
- **Buffer**：尚未处理的输入词；
- **Arcs**：已经建立的依存弧集合。

8.2.2 Arc-standard 动作

最经典的三个动作是：

1. **Shift**: 把 buffer 首词压入 stack;
2. **Left-Arc**: 把栈顶附近两个元素连成左弧, 并弹出 dependent;
3. **Right-Arc**: 把栈顶附近两个元素连成右弧, 并弹出 dependent。

它的本质 可以把它理解为: 通过一串局部动作逐步构造整棵依存树。和 CKY 那种“填表拼区间”的思路不同, transition-based 方法更像“增量式决策过程”。

8.2.3 为什么需要 oracle

训练 transition-based parser 时, 常需要 oracle 告诉系统: 在当前状态下, 哪一个动作是通向金标准依存树的正确下一步。这样才能把解析过程转成分类学习问题。

8.2.4 beam search

如果每一步只贪心选一个最佳动作, 错误可能会一路累积。beam search 会在每一步保留若干候选状态, 降低早期错误导致的灾难性后果。

8.3 句法分析的评价方法

8.3.1 成分句法: Bracketing Precision / Recall / F1

成分句法通常比较系统输出的括号结构与金标准括号结构:

- Precision: 系统给出的括号中有多少是真的;
- Recall: 金标准括号中有多少被系统找回;
- F1: 二者调和平均。

为什么不用简单 accuracy 因为一棵树包含很多局部结构, 部分正确也应该得到部分信用。树结构不是“全对/全错”的单点标签。

8.3.2 依存句法: UAS / LAS

- **UAS** (Unlabeled Attachment Score): head 预测正确的比例;
- **LAS** (Labeled Attachment Score): head 和依存关系标签都正确的比例。

LAS 为什么比 UAS 更严格 因为仅仅连对了 head 还不够，关系类型也得对。例如“主谓”和“动宾”即便 head 一样，语法功能也完全不同。

8.4 这一专题必须会的答题点

1. 能说明 CKY 的状态、初始化和递推；
2. 能说明 CKY 是区间动态规划；
3. 能说清 transition-based parser 的 stack/buffer/arcs；
4. 能列出 Shift / Left-Arc / Right-Arc；
5. 能区分成分句法的 F1 与依存句法的 UAS/LAS。

9 高频综合对比与易错点

9.1 高频综合对比

1. **分类 vs 序列标注**：分类输出一个标签；序列标注输出整条标签序列，因此后者要考虑标签间依赖与全局一致性。
2. **朴素贝叶斯 vs 最大熵**：前者是生成式，靠计数与独立假设；后者是判别式，靠特征与条件似然。
3. **HMM vs CRF**：前者建模联合概率并带发射/转移独立假设；后者直接建模条件概率、全局归一化、特征更灵活。
4. **N-gram vs 神经语言模型**：前者依赖离散计数，固定窗口，稀疏严重；后者通过 embedding 和参数共享缓解稀疏并提升泛化。
5. **显式统计词表示 vs 神经词向量**：前者可解释但高维稀疏；后者低维稠密但解释性较弱。
6. **CFG vs PCFG**：CFG 决定“哪些树合法”，PCFG 进一步决定“哪棵合法树更可能”。
7. **成分句法 vs 依存句法**：前者看短语层次，后者看词间支配关系；评价指标分别偏向括号匹配与 attachment score。

9.2 易错点清单

1. 把序列标注当成逐词独立分类。这会忽视标签间约束，是最基础但也最致命的理解错误。
2. 混淆生成式和判别式。问自己：到底是在建模 $p(x | y)p(y)$ ，还是直接建模 $p(y | x)$ ？
3. 把困惑度当成准确率。PPL 衡量的是概率分布质量，不是 top-1 正确率。
4. 把 PMI 直接当“语义相似度”。它衡量的是词与上下文的关联强度，不等于最终词向量相似度。
5. 只背 Viterbi 名字，不会写状态和递推。算法题这样会丢很多分。
6. 把 CKY 死记成表格技巧。它本质上是区间动态规划。
7. 只记 UAS，不记 LAS。一旦题目强调 relation label，就必须写 LAS。

10 最后冲刺：公式速记与答题模板

10.1 最值得默写的公式

1. 朴素贝叶斯：

$$y^* = \arg \max_y p(y) \prod_i p(x_i | y).$$

2. 对数线性模型：

$$p(y | x) = \frac{\exp(\mathbf{w} \cdot \mathbf{f}(x, y))}{\sum_{y'} \exp(\mathbf{w} \cdot \mathbf{f}(x, y'))}.$$

3. HMM 联合概率：

$$p(x_{1:T}, y_{1:T}) = \pi_{y_1} b_{y_1}(x_1) \prod_{t=2}^T a_{y_{t-1}, y_t} b_{y_t}(x_t).$$

4. Viterbi：

$$\delta_t(j) = \max_i \delta_{t-1}(i) a_{ij} b_j(x_t).$$

5. 链式法则与 N-gram：

$$p(w_{1:T}) = \prod_{t=1}^T p(w_t | w_{1:t-1}), \quad p(w_t | w_{1:t-1}) \approx p(w_t | w_{t-n+1:t-1}).$$

6. N-gram MLE：

$$p(w | h) = \frac{\text{Count}(h, w)}{\text{Count}(h)}.$$

7. 困惑度：

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_t \log p(w_t | h_t)\right).$$

8. PMI：

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)}.$$

9. CRF：

$$p(y | x) = \frac{\exp(\text{Score}(x, y))}{\sum_{y'} \exp(\text{Score}(x, y'))}.$$

10. PCFG：

$$p(T) = \prod_{r \in T} p(r), \quad p(A \rightarrow \beta) = \frac{\text{Count}(A \rightarrow \beta)}{\sum_{\beta'} \text{Count}(A \rightarrow \beta')}.$$

11. CKY：

$$\text{Chart}[i, j, A] = \max_{k, A \rightarrow BC} p(A \rightarrow BC) \cdot \text{Chart}[i, k, B] \cdot \text{Chart}[k + 1, j, C].$$

10.2 最稳妥的比较题模板

若题目要求“比较 A 和 B”，建议按下列顺序作答：

1. **任务/输出对象**：它们各自解决什么问题；
2. **建模对象**：它们建模的是联合概率、条件概率，还是直接结构；
3. **核心假设**：例如条件独立、马尔可夫假设、全局归一化；
4. **训练方式**：计数、极大似然、条件似然、梯度优化；
5. **解码方式**：局部分类、Viterbi、CKY、transition actions；
6. **评价方式**：accuracy/F1/PPL/UAS/LAS；
7. **优劣与适用场景**。

10.3 最稳妥的算法题模板

1. 先写状态定义；
2. 再写初始化；
3. 然后写递推/动作规则；
4. 最后补回溯/输出恢复与复杂度。

10.4 最稳妥的简答题模板

如果被问“为什么某模型有某缺陷/优势”，建议按下面的逻辑组织：

1. 先说模型做了什么关键假设；
2. 再说这个假设带来了什么计算优势；
3. 最后说这个假设因此忽略了什么真实语言现象。

11 整理说明

这份期中专题复习笔记依据当前已整理的 FNL P 全讲义版本再次重组而成，专门围绕 2026 年 4 月 22 日这次用户明确给出的考试范围展开。它保留了课程中的主模型、核心公式、经典算法与评价指标，但删去了大量不直接服务于期中复习的扩展材料，使结构更接近考前冲刺时的“专题串讲稿”。